

I/O Error on a Datafile:

Instance Crash or Datafile Offline?

The arrival of the multitenant databases makes the instance availability even more critical, since it can now run multiple databases. What do you think happens when there is an error writing to a datafile? There was a major change in 11.2.0.2 which was a little unnoticed because we don't remove datafiles frequently. Except of course when we install a new Oracle infrastructure at a customer because we always test backup/recovery scenarios before going into production. Let's see what happens when we delete a datafile in RAC and especially multitenant 12c.

Franck Pachot, dbi-services ♠

Remember, we used to have the following scenarios in mind:

- Loss of a member of the controlfile → instance crash
- Loss of a member of redo log → message in alert. log. If there is only one member then we will have to restart the instance (with loss of transactions).
- Loss of a system datafile → instance crash
- Loss of a non-system datafile → the datafile goes offline

The last one is wrong. That was before 11.2.0.2 and here is an example in 12c:

```
RMAN> host "rm -f /u02/oradata/CDB/PDB1/PDB1_users01.dbf";
host command complete
```

```
RMAN> alter system checkpoint;
RMAN-00571: =====
RMAN-00569: ===== ERROR MESSAGE STACK FOLLOWS =====
RMAN-00571: =====
RMAN-00601: fatal error in recovery manager
RMAN-03004: fatal error during execution of command
ORA-01092: ORACLE instance terminated. Disconnection forced
RMAN-03002: failure of sql statement command at 02/19/2015 22:51:55
ORA-03113: end-of-file on communication channel
Process ID: 19135
Session ID: 357 Serial number: 41977
ORACLE error from target database:
ORA-03114: not connected to ORACLE
```

I've run the 'alter system' from RMAN because we can run any SQL from RMAN in 12c. But the point here is that my instance is stopped. All services (from all PDBs) are stopped, even if nobody cares about the USERS tablespace of PDB1.

Before 11.2.0.2, this scenario would simply have put the USERS tablespace offline, and most of the users would have been able to continue their work. Which one do you prefer? Let's think about it.

Any reason to put the datafile offline?

We are so used of the previous behaviour that we often find it normal to only put the datafile offline when there is no need to stop the whole instance. But is it the right choice to let the instance run when it is impossible to checkpoint all the datafiles? Why not just stop the instance, in the same way as when it is a system datafile which is missing? And why this behaviour only when we are in ARCHIVELOG mode?

The reason is that when it is a non-system tablespace, and when we are in ARCHIVELOG mode, and when the problem is media failure, then we are able to restore and recover the datafile, and put it back online, without stopping the instance.

Why all those conditions? Because if it is a system tablespace, then the database cannot remain open because SYSTEM must always be online. If you are not in archive log mode, then the datafile cannot be recovered anyway. And if the whole storage is offline then nobody can work anyway.

But if you have consolidated several applications into one database, then the applications that do not use the missing tablespace can continue to be online. Thus, the primary choice was to put the failed datafile offline and keep the other applications available.

This choice was valid for many installations in the past, but today:

- What is the probability of losing a single datafile rather than the entire storage? Today everything is stripped on the storage. Human error? If you are in ASM, you don't touch the datafiles so no risk to delete one by mistake.
- Do you really have many users who can continue to work when a datafile is offline? We often have only one application per database. All data are linked. We'll see the multitenant case later.
- Don't you have some failover mechanisms which let you open the full service again in few minutes in case of any failure? Much less than the time it takes to recover a datafile manually. If the instance does not see the datafile, but it is still there, then RAC maintains the access from another instance. If the file is lost, DataGuard (or Dbvisit standby when in Standard Edition) can open a copy of the database quickly and automatically when configured with fast start failover.

So, rather than keeping a datafile offline until we do the manual operation of restoring and recovering it and having the tablespace inaccessible from any RAC node during that time, don't you prefer to stop the instance automatically and have the database available on another server that can see the file?

11gR2 – Patchset 2

This is the idea of 'bug 7691270', better considered as an 'enhancement request', coming with 11.2.0.2 and changing that behaviour that we know from a long time: any error when writing to any datafile during checkpoint will triggers the immediate (well, it's an abort) shutdown of the database. And it is still the case in 12c as we have seen in the example above. This is also the reason why the previous behaviour - datafile offline - was implemented only in archive log mode to be sure you can do the recovery of the datafile. In NOARCHIVELOG mode, redo would soon be overwritten in a future log switch and will be lost. The instance crash is the only way to avoid having to generate more redo. We will talk about pluggable databases later.

Note that this applies only when writing the 'dirty buffers' at a checkpoint. Errors in reading, or writing in direct-path, are logged in the alert.log but do not change the status of the datafile nor the state of the instance.

datafile_write_errors_crash_instance

This new behaviour is enabled by default in 11.2.0.2 by the following parameter:

```
_datafile_write_errors_crash_instance = true
```

True is the default value. We can return to the previous mode with:

```
alter system set "_datafile_write_errors_crash_instance"=false
```

The new behaviour is best suited to what we usually expect today. But if you have consolidated multiple applications into one database, and you don't have a fast failover mechanism ensuring high availability and you think that it is possible to lose one datafile without losing the access to whole storage, then you may prefer to fall back to the previous behaviour and set that parameter to false.

Two reminders:

- This is an undocumented parameter, it is a good idea to ask Oracle Support if there is no problem to set it in your context.
- If you are not in ARCHIVELOG mode, then no matter, as you don't care about your data...

RAC

This change was primarily motivated by the fact that more and more databases are in high availability. Putting a datafile offline in RAC makes it inaccessible to all instances. But very often the write error does not come from the absence of the file, but from a failure in the access from one node. So it is better to let other instances access to the file.

If, for whatever reason, you prefer to have the datafile offline in RAC with "`_datafile_write_errors_crash_instance`" set to false, then make sure to be in 11.2.0.4 or 12c, because of bug 13745317 where the agent stops all the cluster instances when a datafile is offline.

12c Multitenant

We have seen above that one good reason to put the datafile offline can be when we have multiple applications into one

database, which was not very frequent today in 11g – hence the decision to stop that behaviour in 11.2.0.2.

But 12c brought a new way to consolidate with multitenancy. So, do you still want to stop a whole CDB instance at the first write failure occurring on any user datafile which belongs to only one pluggable database?

If the High Availability is ensured by DataGuard in fast-start failover configuration with an observer, then yes, it is probably better to failover all sessions to the site that has no I/O issue. If you are in RAC, it is possible to relocate only the services that are concerned by the lost datafile. So it depends if you have an SLA defined for the whole CDB or if it is defined at PDB level.

Then, in multitenant, there is a good chance that you prefer to stop only one PDB. This is done by accepting to put the datafile offline:

```
alter system set "_datafile_write_errors_crash_instance"=false
```

But this not sufficient. That concerns only the user datafiles. But you probably don't want to stop the whole CDB even if you lose a system datafile of one PDB. I opened a bug about that (19001390), which is fixed in the latest PSU.

Pluggable database

Well, with that bug fixed, the CDB instance will not stop. But this is not yet enough. If we have lost a datafile, then we have to restore it. And when you lose a system datafile, then you must do a shutdown abort because the system tablespace cannot be offlined. In non-CDB, no problem: the instance is crashed, there is no risk to overwrite some redo.

But doing a 'shutdown abort' for a PDB is completely different. The online redo logs are protecting the whole CDB and they will be overwritten at the next log switches. By default, Oracle prevents that. You cannot 'shutdown abort' a pluggable database. You need to wait until you can stop the whole CDB before you can restore and recover the missing file.

This is a major availability problem if we cannot switchover to a standby database. If we are in archivelog mode, then we may accept to do that 'shutdown abort' of the PDB because even if the online redo logs are overwritten we can recover from the archived redo logs. We can allow that with:

```
alter system set "_enable_pdb_close_abort"=true
```

If we are not in archivelog mode, then that's a high risk. So even with that parameter to true, the PDB cannot be offlined. We can force to possibility to do it with:

```
alter system set "_enable_pdb_close_noarchivelog"=true
```

But the risk then is to lose the PDB when we don't have the redo in online redo logs. So that setting is acceptable only if we can re-create the PDB (a clone in dev for example).

These two parameters can be defined at each PDB, but must be

defined in CDB if you do not want that the instance is stopped at the checkpoint.

Example

To summarise, here is the behaviour after deleting the system datafile of a PDB when the CDB (with latest PSU) has the following parameters (which are not the defaults):

```
"_datafile_write_errors_crash_instance"=false
"_enable_pdb_close_abort"=true
```

I will use the Recovery Advisor to see the error:

```
RMAN> list failure;
```

Database Role: PRIMARY

List of Database Failures

=====

Failure ID	Priority	Status	Time Detected	Summary
897	CRITICAL	OPEN	23-MAR-15	System datafile 8: '/u02/oradata/CDB/PDB/datafile/ol_mf_system_bk04nchr_.dbf' is missing

This datafile belongs to the PDB system tablespace. Thanks to the previous setting, the instance has not been stopped at the checkpoint.

```
RMAN> advise failure;
```

Database Role: PRIMARY

List of Database Failures

=====

Failure ID	Priority	Status	Time Detected	Summary
897	CRITICAL	OPEN	23-MAR-15	System datafile 8: '/u02/oradata/CDB/PDB/datafile/ol_mf_system_bk04nchr_.dbf' is missing

analyzing automatic repair options; this may take some time
allocated channel: ORA_DISK_1
channel ORA_DISK_1: SID=50 device type=DISK
analyzing automatic repair options complete

Mandatory Manual Actions

=====

no manual actions available

Optional Manual Actions

=====

1. If file /u02/oradata/CDB/PDB/datafile/ol_mf_system_bk04nchr_.dbf was unintentionally renamed or moved, restore it
2. **Automatic repairs may be available** if you shutdown the database and restart it in mount mode

Automated Repair Options

=====

Option Repair Description

1 **Restore and recover datafile 8**

Strategy: The repair includes complete media recovery with no data loss

Repair script: /u01/app/oracle/diag/rdbms/cdb/CDB/hm/reco_2804473397.hm

The Recovery Advisor suggests the restore/recover:


```

RMAN> repair failure;

Strategy: The repair includes complete media recovery with no
data loss
Repair script: /u01/app/oracle/diag/rdbms/cdb/CDB/hm/
reco_2804473397.hm

contents of repair script:
# restore and recover datafile
sql 'PDB' 'alter database datafile 8 offline';
restore ( datafile 8 );
recover datafile 8;
sql 'PDB' 'alter database datafile 8 online';

Do you really want to execute the above repair (enter YES or
NO)? YES
executing repair script

sql statement: alter database datafile 8 offline

```

However:

```

RMAN-00571: =====
=====
RMAN-00569: ===== ERROR MESSAGE STACK FOLLOWS
=====
RMAN-00571: =====
=====
RMAN-03002: failure of repair command at 03/23/2015 14:22:34
RMAN-03015: error occurred in stored script Repair Script
RMAN-03009: failure of sql command on default channel at
03/23/2015 14:22:34
RMAN-11003: failure during parse/execution of SQL statement:
alter database datafile 8 offline
ORA-01541: system tablespace cannot be brought offline; shut
down if necessary

```

But unfortunately, the pluggable database has to be closed and Recovery Advisor forgot to do this step. Therefore we have to do it manually:

```

RMAN> alter pluggable database PDB close;

Statement processed

```

Which is possible thanks to “**_enable_pdb_close_abort**=true (I am in archivelog mode).

Then the recovery can now be done:

```

RMAN> repair failure;

...
media recovery complete, elapsed time: 00:00:01
Finished recover at 23-MAR-15

sql statement: alter database datafile 8 online
repair failure complete

```

We still have to open the PDB ourself.

As there has been a shutdown abort, we must also run a recover on all files to be sure they are consistent:

```

RMAN> alter pluggable database PDB open;

RMAN-00571: =====
=====
RMAN-00569: ===== ERROR MESSAGE STACK FOLLOWS
=====
RMAN-00571: =====
=====
RMAN-03002: failure of sql statement command at 03/23/2015
14:40:27
ORA-01113: file 9 needs media recovery
ORA-01110: data file 9: '/u02/oradata/CDB/datafile/o1_mf_sysaux_
bjdxygdj_.dbf'

```

Once again this is not done by Recovery Advisor so we do it manually:

```

RMAN> recover pluggable database PDB;

Starting recover at 23-MAR-15
using channel ORA_DISK_1

starting media recovery
media recovery complete, elapsed time: 00:00:00

Finished recover at 23-MAR-15

RMAN> alter pluggable database PDB open;

Statement processed

```

This is not all automated, but it is still possible without impacting the availability of other pluggable databases. Of course, you must have a good monitoring of the alert.log: we will need the archivelogs, so it is better not to wait many hours.

Conclusion

I detailed the different scenarios and settings because you have to understand the consequences in terms of availability. There is a default behaviour, that changed to adapt most frequent situations, but there cannot be a general case suited for every situation. High availability is achieved differently at each site (RAC, DataGuard with or without observer, Cold failover, vMotion...). Service availability requirements, data availability are specific to each application. What you need in dev and test is different on production.

It is always better to keep default parameters. But in some cases we can prefer to set the parameter “**_datafile_write_errors_crash_instance**” to false. And, in that case, if we are in archivelog mode, I recommend to apply the latest PSU and set “**_enable_pdb_close_abort**” to true in order to be able to restore the missing file without stopping the whole CDB.

In all cases, it is mandatory to have good monitoring and get an alert as soon as the alert.log reports a write error to a datafile. The longer we wait and the more difficult the recovery will be. Recovery Advisor can help, but you still need to understand recovery as you may have to do it manually.

Last remark about datafile loss. Today, many companies consider that the availability of files is provided by the storage, with RAID levels ensuring mirroring, SAN synchronised, etc. Unfortunately, they often forget to consider the human error. The loss of

SUMMER 15

Technology: Franck Pachot

a file can simply come from a mistake, a bug in a script, etc. And in this case, you will be happy to be able to quickly restore and recover and put the datafile back online without having to stop all applications.

For the same reason, from our experience with customers, we always recommend to multiplex controlfiles and redo log members even when they are mirrored at storage level. ■



ABOUT THE AUTHOR

Franck Pachot 
Senior Consultant, dbi services

Franck Pachot is senior consultant at dbi services in Switzerland. He has 20 years of experience in Oracle databases, all areas from development, data modeling, performance, administration, and training. Oracle Certified Master and Oracle ACE, he tries to leverage knowledge sharing in forums, publications, presentations.

Blog: <http://blog.pachot.net>



www.linkedin.com/in/franckpachot

@FranckPachot

MEET A VOLUNTEER: JASON

Name: Jason Arneil 
Job Title: Infrastructure Architect
Company: Accenture Enkitech Group

Tell us about yourself in 50 words
Experienced IT guy, DBA and System Administrator - combined both those roles for most of my career.

What are your interests (inside and outside of work)?
Football, books and politics.

When did you start volunteering with UKOUG?
I started presenting around 10 years ago.

Why did you want to become a UKOUG volunteer?
I wanted to increase my exposure within the Oracle community.

What volunteering activities do you undertake?

I'm involved in presenting mostly, although I've also helped organise the engineered systems events and the annual conference agenda.

Which volunteer contribution are you proudest of?

It has to be the Engineered Systems Event in London in 2013. I thought it was ace and to have helped set that up was a great feeling.

What's most rewarding about volunteering?

It really does increase your visibility in the community and you do get a



lot of contacts as a result.

What experience with UKOUG influenced you the most?

The annual conference - I first attended in 2000 and I have gone every year since. It's really improved my knowledge.



@jarneil

Blog: <http://jarneil.wordpress.com>

Get Involved!

Whether you can commit to 5 minutes or 5 days, we can use your help.

If you're enthusiastic about Oracle and believe in working co-operatively for the benefit of the whole community, we want to hear from you!

Volunteers are at the heart of everything we do at UKOUG and are integral to

our success, ensuring we continue to innovate and lead the way. In all tasks, our volunteers work alongside experienced staff and other volunteers. You will meet other professionals, broaden your knowledge, develop new interests and have lots of fun along the way.

Simply complete the online form and a member of the team will be in touch:
www.ukoug.org/icanhelp



www.ukoug.org